

AI Project

Flappy Bird Player Agent

Nicolas MARMANN
1^{re}CI1



Lycée des Arts et Métiers

Section I – Informatique et Communication

Table of Contents

TABLE OF CONTENTS.....	2
TABLE OF FIGURES	3
I. PROJECT OVERVIEW.....	4
I.1. Type of Project.....	4
I.1.a. Use of AI Tools.....	4
I.1.b. Reason for Using AI	4
I.2. Project Idea / Goal	5
II. TECHNOLOGY & STACK.....	6
II.1. Requirements	6
II.1.a. Programming Language	6
II.1.b. Libraries	6
II.1.c. Game Source Code.....	6
II.2. Development Environment.....	6
II.2.a. Code Editor.....	6
II.2.b. Virtual Environment.....	6
III. DEPENDENCIES & INSTALLATION	7
III.1. Setup.....	7
III.1.a. Python Version	7
III.1.b. Create a Virtual Environment	7
III.1.c. Activate the Virtual Environment	7
III.1.d. Install Dependencies	8
III.2. Run the Game	8
III.2.a. Game Controls.....	8
III.3. Run the AI Training.....	9
III.3.a. AI Controls.....	9
III.4. Run the Demo of the Best Bird.....	10
III.4.a. Demo Controls.....	10
III.4.b. Best bird.....	10
IV. IMPLEMENTATION.....	11
IV.1. AI Training (ai.py)	11
IV.1.a. Input data for the neural network	11
IV.1.b. Fitness calculation.....	11
IV.1.c. Saving the best bird.....	12
IV.2. Demo of the AI (demo_best_bird.py)	12
IV.3. Configuration of the NEAT Algorithm.....	12

V.	RESULTS & CONCLUSION	13
V.1.	Did it work?.....	13
V.2.	First Results	13
V.3.	Performance.....	14
VI.	LESSONS LEARNED	15
VII.	NEXT STEPS	16
VII.1.	Improvements	16
VII.2.	New Features.....	16
VII.3.	Expansion Ideas / Opportunities	16
VIII.	REFERENCES AND APPENDICES	17
VIII.1.	Bibliography	17
VIII.2.	Sources of Illustrations	17

Table of Figures

Figure I.1 – Flappy Bird	5
Figure III.1 – Flappy Bird original game	8
Figure III.2 – AI Training.....	9
Figure III.3 – AI Demo.....	10

I. Project Overview

I.1. Type of Project

This project was created from scratch.

This means that most of the code and implementation were developed by myself, and only the source code of the game was taken directly from the web.

I used an existing open-source Flappy Bird project as a base reference for the game code only, not for the AI system. The AI logic, training process, and related features were implemented by myself and adapted to meet the goals of the project.

The main focus of this project was to:

- Implement an artificial intelligence (AI) model
- Train the AI to play the game automatically

I.1.a. Use of AI Tools

During this project, I used AI tools to support my learning and development process.

The AI tools were used mainly for:

- Understanding how the game code works
- Learning new programming concepts
- Fixing errors and debugging problems

The AI tools were not used to automatically generate the full project (which would not have been possible anyway).

All important decisions, implementations, and testing were done by myself.

I.1.b. Reason for Using AI

I used AI tools because:

- Python was not taught in school and I did not have prior experience with this programming language
- AI programming was only briefly introduced during the course
- The project required concepts beyond the course material

AI tools were used responsibly and as a support tool, not as a replacement for my own work.

I.2. Project Idea / Goal

The main idea of this project is to train an artificial intelligence (AI) to play the game Flappy Bird automatically.

In the normal version of the game, a human player controls the bird and decides when to jump to avoid obstacles (pipes). In this project, the AI player agent takes over this role and makes decisions on its own based on the game situation.

At the beginning of the training process, the AI starts with random behaviour and does not know how to play the game. Later, the AI improves by learning from previous attempts. Birds that survive longer and pass more obstacles are considered better and are used to create the next generation.

The goal of this process is to improve the performance of the AI step by step until it can play the game successfully without human input. Another important goal of the project is to save the best-performing AI bird. The best bird is stored in a file so that it can be loaded later and used to demonstrate the trained AI without running the full training process again. This also allows a human player to play against the trained AI and see who does better.



Figure I.1 – Flappy Bird

II. Technology & Stack

II.1. Requirements

The following technologies and software are required to run this project.

II.1.a. Programming Language

Python 3.12.3 is used to run the game and the AI training process.

II.1.b. Libraries

The required libraries for this project are listed in the `requirements.txt` file:

- pygame ($\geq 2.1.0$) → used to run the game and display graphics
- neat-python (≥ 0.92) → used to train the AI using the NEAT algorithm

These libraries are installed automatically using:

```
pip install -r requirements.txt
```

II.1.c. Game Source Code

The game source code used in this project comes from an open-source GitHub repository: FlapPyBird – sourabhv ([GitHub](#))

This repository provides the original Flappy Bird game implementation, which was used as the starting point for the game environment. The AI system and training logic were implemented separately as part of this project.

II.2. Development Environment

II.2.a. Code Editor

Visual Studio Code (VS Code) was used to write, edit, and run the Python code.

II.2.b. Virtual Environment

A virtual environment (Python venv) was used to manage dependencies and keep the project setup isolated from other projects.

The virtual environment can be created using:

```
python -m venv venv
```

III. Dependencies & Installation

This section explains how to install the required dependencies and how to run the project. Following these steps should allow the project to run correctly on a new device.

III.1. Setup

III.1.a. Python Version

The project was developed and tested using `Python 3.12.3`.

It is recommended to use the same version or a compatible version of `Python 3`.

You can check your Python version using:

```
python --version
```

If Python is not installed or the version is too old, download and install the latest version of Python from the official website: <https://www.python.org/>

After installing Python, restart the terminal and check the version again.

III.1.b. Create a Virtual Environment

A virtual environment is used to keep the project dependencies separate from other Python projects.

To create a new virtual environment, run the following command in the project folder:

```
python -m venv venv
```

This command creates a folder named `venv` that contains the isolated Python environment.

III.1.c. Activate the Virtual Environment

Before installing dependencies, the virtual environment must be activated.

To do this, execute the following command in the project directory:

```
.\venv\Scripts\Activate.ps1
```

After activation, the terminal should show `(venv)` at the beginning of the line.

III.1.d. Install Dependencies

The required libraries for this project are listed in the file `requirements.txt`.

Install all dependencies using the following command:

```
pip install -r requirements.txt
```

This will install the required libraries:

- pygame
- neat-python

III.2. Run the Game

To start the Flappy Bird game without AI, run:

```
python -m src.flappy
```

III.2.a. Game Controls

- SPACE – start the game
- SPACE – flap
- ESC – quit

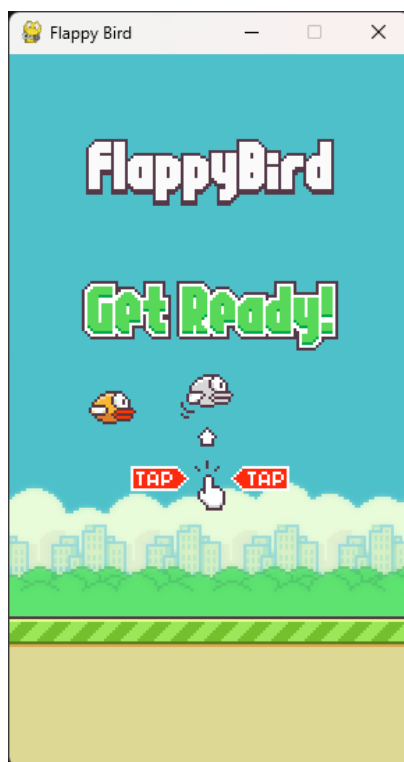


Figure III.1 – Flappy Bird original game

III.3. Run the AI Training

To train the AI and start the learning process, run:

```
python ai.py
```

III.3.a. AI Controls

- 1 – AI mode
- 2 – Human + AI mode
- R – restart
- ESC – quit
- SPACE – flap

During training, the AI will play the game repeatedly and improve its performance step by step.

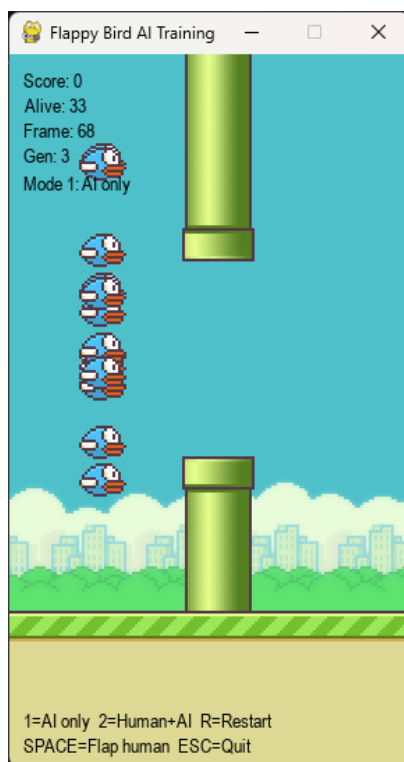


Figure III.2 – AI Training

III.4. Run the Demo of the Best Bird

After training, the best-performing AI can be demonstrated using:

```
python demo_best_bird.py
```

This loads the saved best bird and runs the game using the trained AI.

III.4.a. Demo Controls

- 1 – AI mode
- 2 – Human + AI mode
- R – restart
- ESC – quit
- SPACE – flap

This also allows a human player to play against the trained AI and compare results.

III.4.b. Best bird

To run the demo correctly, the trained bird that should be used is the best-performing one from the training process. This file is created automatically during training when running the program (`ai.py`). During the training process, the AI plays the game repeatedly and whenever a better-performing bird is found, its data is saved to a `.json` file.

In this project, the best bird after training is saved as: `best_genome_50_gens.json`

This file represents the best-performing AI after 50 generations (> 3 hours) of training and is the version intended for demonstration.

Before running the demo, this file needs to be renamed to: `best_genome.json`

The demo program (`demo_best_bird.py`) automatically looks for a file with this exact name to load the trained bird (it is hard-coded in the demo program).

In addition to the `.json` file, the training process also creates a file with the extension `.pkl` (pickle file). This file stores the same trained bird in Python's internal format and it is included as a fallback option for loading the trained AI if needed.

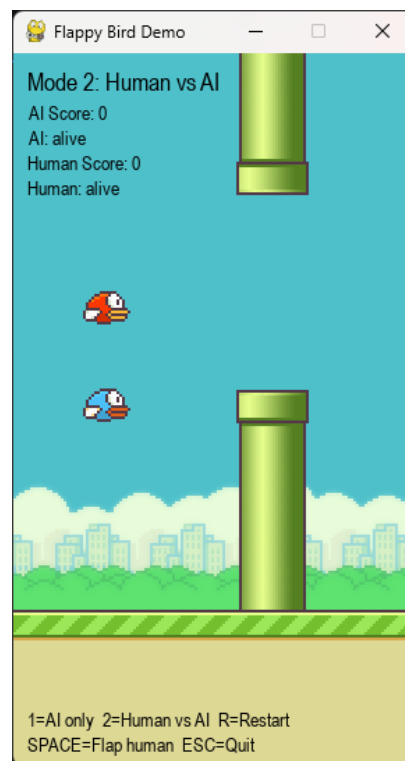


Figure III.3 – AI Demo

IV. Implementation

IV.1. AI Training (ai.py)

The file `ai.py` contains the main logic for training the artificial intelligence.

Its purpose is to train multiple birds to play Flappy Bird using the NEAT algorithm. During training, many birds play the game at the same time. Each bird uses a neural network to decide when to flap.

The training process follows these main steps:

1. Create a population of birds
2. Let all birds play the game
3. Evaluate how well each bird performs
4. Select the best-performing birds
5. Create a new generation from the best birds
6. Repeat the process

This process continues for multiple generations until a good-performing bird is found.

IV.1.a. Input data for the neural network

Each bird receives information from the game environment. This information is used as input for the neural network. The input values include:

- The bird's vertical position
- The distance to the next pipe
- The position of the gap between pipes

Based on this information, the neural network decides to flap (jump) or do nothing. This decision is made automatically during gameplay.

IV.1.b. Fitness calculation

During training, each bird receives a fitness score. The fitness score measures how well the bird performs in the game. The fitness increases when:

- The bird survives longer
- The bird passes obstacles
- The bird avoids collisions (obstacles & ground)

Birds with higher fitness scores are considered better and are used to create the next generation. This allows the AI to improve its behaviour.

IV.1.c. Saving the best bird

When a new best-performing bird is found, it is automatically saved to a file. The best bird is exported as: `best_genome.json`

This file contains the trained neural network data, including:

- An identifier
- Fitness score of the bird
- The structure of the neural network
- Neuron parameters (such as bias and activation function)
- The connection weights between neurons

Saving the best bird allows the trained AI to be reused later without running the full training process again.

IV.2. Demo of the AI (`demo_best_bird.py`)

The file `demo_best_bird.py` is used to demonstrate the trained AI. Instead of training the AI again, this program loads the saved bird from the `.json` file and runs the game using the trained model.

This makes it possible to:

- Show the final result
- Demonstrate the AI performance
- Compare the AI with a human player

The program automatically loads the file: `best_genome.json`

The file name must match exactly because it is defined directly in the program code.

IV.3. Configuration of the NEAT Algorithm

The behaviour of the AI training process is controlled by the file: `neat-config.txt`. This configuration file defines how the neural network and training process work.

Important settings in this file include:

- Number of inputs
- Number of outputs
- Population size
- Mutation rates (how often changes are applied to the neural network)
- Fitness settings (how well the AI performs)

Changing these values can affect how fast the AI learns and how well it performs.

V. Results & Conclusion

V.1. Did it work?

Yes, the project worked successfully in the end, but there were several issues during development.

At the beginning, the system did not work correctly. There were multiple errors, and in some cases the game did not launch at all. These problems were mainly related to missing dependencies, incorrect configuration, and setup issues.

Through testing and debugging, these problems were gradually resolved. Once the environment was set up correctly, the game and the AI training process were able to run without errors.

After resolving the initial issues, the system became stable and the AI could be trained successfully. The best-performing bird could then be saved and demonstrated using the demo program.

Overall, the project achieved its main objective of creating a working AI that can learn to play Flappy Bird automatically.

V.2. First Results

After the technical problems were resolved, the training process was able to start. However, the first results were not successful. At the beginning of the learning process, the AI did not learn anything useful. The birds performed random jumps without making meaningful decisions, which caused them to fail quickly. In some cases, the birds simply fell straight down to the ground and died, while in other cases they jumped directly into the upper edge of the game window. This behaviour showed that the AI was not using the correct information from the game. The input values for the neural network were then adjusted and improved.

The AI now uses important data from the game environment, such as:

- The height of the bird
- The position of the gap between the pipes
- The distance to the next obstacle

After these changes, the AI started to make more consistent decisions. The birds survived longer, passed more obstacles, and showed clear improvement from generation to generation. These improvements confirmed that the training process was working correctly.

V.3. Performance

After running the training for 50 generations, the AI reached a stable performance level. The training process took more than 3 hours. During this time, the AI gradually improved its behaviour and became better in avoiding obstacles.

The best-performing bird was saved as: `best_genome_50_gens.json`

This file represents the final trained AI and can be used to demonstrate the system without running the training process again.

(Before running the demo, this file must be renamed to: `best_genome.json`)

The trained AI is able to:

- Survive for long periods of time (I never observed the bird failing)
- Pass multiple pipes
- Make consistent decisions
- Avoid collisions more effectively than earlier generations

The system also allows a human player to play against the trained AI and compare performance. Overall, the results show that the AI training process was successful and that the implemented system works.

VI. Lessons Learned

During this project, I learned some practical lessons about programming, artificial intelligence, and working on real software projects.

One of the first things I realised is how important documentation is. While doing research and looking for example projects, I noticed this problem many times. Many projects on the internet look complete and good, but they do not always work immediately when you try to run them. Clear documentation and correct setup instructions are essential if other people are expected to use the project. However, in many cases the documentation is not good or even non-existent.

Another lesson was that most of the time in software development is spent debugging, not writing new code. Many errors occurred during development, especially at the beginning when the game did not launch. Solving these problems required patience, systematic testing of different solutions, and carefully reading error messages. This process helped improve problem-solving skills and understanding of the code.

I also learned that training an AI takes a long time. Even after the system was working, the training process itself required several hours to run. At the beginning, there was no noticeable progress even after many hours of training. Later, it was able to learn much faster. This showed that machine learning is not only about writing code for a good AI model, but also about waiting for the system to learn through repeated attempts.

In addition, developing an AI model is a long process and the first results are usually very bad. At the start of the training process, the AI performed random jumps and failed quickly. Only after adjusting the input values and configuration, the AI began to improve and behave in a more intelligent way and was actually able to learn something from previous attempts.

Another lesson was about the limitations of AI tools / chatbots. While tools like ChatGPT can be helpful for understanding concepts and fixing errors, they are not able to solve every problem automatically. Many issues still required manual testing, debugging, and personal understanding of the code.

Finally, the project showed that training a new artificial intelligence can take a significant amount of time and computing resources (the battery of my laptop was nearly dead after the 50-generation training).

Overall, this project provided practical experience with artificial intelligence, Python programming, debugging, and system configuration, and improved my understanding of how machine learning systems work in real-world applications.

VII. Next Steps

VII.1. Improvements

One possible improvement could be to optimise the training process to reduce the total training time. Currently, training the AI takes several hours, especially when running many generations. Improving the performance of the program itself or using more efficient hardware could reduce the required time.

Another improvement would be to experiment with different configuration parameters in the NEAT configuration file. Changing values such as population size, mutation settings, or fitness calculation could improve the learning behaviour of the AI and lead to more stable or better results.

VII.2. New Features

One possible useful feature would be to allow the user to adjust training settings directly from the program. For example, the number of generations or training speed could be configured without changing the source code.

It would also be interesting to add different difficulty levels to the game. For example, the speed of the pipes or the size of the gap could be adjusted to test how well the AI adapts to new conditions.

VII.3. Expansion Ideas / Opportunities

In the future, the project could be expanded beyond the current Flappy Bird game.

One possible expansion would be to implement a graphical interface to visualise the neural network of the bird in real time. For example, the program could display how the input values change and how the network makes decisions during gameplay. This would help to better understand how the AI works internally, but it would require learning more about visualisation and graphical programming.

Another opportunity would be to implement automatic saving and comparison of multiple training runs. For example, the system could store several trained birds with different settings and compare their performance. This would allow more systematic testing and experimentation.

Finally, another possible expansion would be to create a web-based version of the project. The trained AI could be demonstrated directly in a browser.

VIII. References and appendices

VIII.1. Bibliography

- Source code of the game: <https://github.com/sourabhv/FlapPyBird>
(accessed on 23.02.2026)

VIII.2. Sources of Illustrations

- Cover page illustration (page 1):
Garmin Connect IQ – “*Flappy Bird*”
<https://apps.garmin.com/apps/d62e2a7f-dbde-4ab6-9695-c1ecafd20f9a>
(accessed on 10.04.2026)
- Gematsu – “*Flappy Bird returning in expanded form for web browsers, iOS, and Android*” – published on 12.09.2024
<https://www.gematsu.com/2024/09/flappy-bird-returning-in-expanded-form-for-web-browsers-ios-and-android>
(accessed on 10.04.2026)